



Aula 10 - ORM Parte 2: Relacionamentos com o Eloquent - *HasManyThrough* e *ManyToMany*

IFSUL | CSTSI | Prof. Gill Velleda Gonzales

LICENÇA: Creative Commons — Attribution-ShareAlike 4.0 International
— CC BY-SA 4.0

Nesta aula, continuaremos a modelagem do sistema com os diferentes tipos de relacionamentos que podemos implementar através do Eloquent ORM. Veremos como criar relações intermediárias de várias níveis hierárquicos em relações 1:N, através de extensões do Eloquent. Seguiremos abordando consultas com o Eloquent, assim como a camada de Query/Builder da API do Laravel. Implementaremos, também no banco de dados de Produtos, uma relação Many-to-Many (N:N) com Promoção.

Repositório:

https://github.com/g1ll/cstsi_bda_20232_firebase_mongodb

Versões:

```
...correcoes/20231/aula_05 <?php 8.2.5 } correcao-aula-08-orm-eloquent-relationship-parte-2
$ php -v;composer -V; php artisan -V
PHP 8.2.5 (cli) (built: Apr 14 2023 04:26:42) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.2.5, Copyright (c) Zend Technologies
    with Zend OPcache v8.2.5, Copyright (c), by Zend Technologies
Composer version 2.5.5 2023-03-21 11:50:05
Laravel Framework 10.5.1
```

Continuidade da [Aula 8](#).

Relação Intermediárias (*hasManyThrough*)

Para Filtrar Fornecedores de uma determinada Região, até o momento, precisamos fazer o seguinte uso do Eloquent:

```
Fornecedor::whereHas('estado',fn($q)⇒$q→whereHas('regiao',fn($q)⇒$q→
where('nome','Sul'))))→get()
```

Ou seja, precisamos primeiro passar pela relação de Fornecedor com Estado e então, a partir de Estado acessar a relação com a Região.

Podemos simplificar utilizando o método **hasManyThrough** do Eloquent, definindo que Região possui uma relação de **hasMany** com Fornecedor, através de (**Through**) Estado;

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Regiao extends Model
{
    use HasFactory;
```

```

protected $table = 'regioes';
protected $fillable = ['nome'];

public function estados(){
    return $this->hasMany(Estado::class);
}

public function fornecedores(){
    return $this->hasManyThrough(Fornecedor::class, Estado::class);
}
}

```

Agora, com a inserção do método **fornecedores** que utiliza o **hasManyThrough**, podemos fazer a seguinte chamada a partir da model Região:

```
Regiao::where('nome','Sul')->with('fornecedores')->get();
```

Será retornada a **Região Sul** com uma **EloquentCollection** de Models de **Fornecedor**.

Como poderíamos retornar então os produtos de uma determinada Região? Neste caso teremos que usar um pacote externo, pois o Eloquent não suporta mais de um nível intermediário.

Vamos então instalar o seguinte pacote:

```
https://github.com/staudenmeir/eloquent-has-many-deep
```

Rodamos então o comando do composer:

```
composer require staudenmeir/eloquent-has-many-deep:"^1.7"
```

Na classe Model de Região adicionamos mais uma Trait **|Staudenmeir\EloquentHasManyDeep\HasRelationships** , e criamos o método

produtos utilizando o método ***hasManyDeep***:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Regiao extends Model
{
    use HasFactory, \Staudenmeir\EloquentHasManyDeep\HasRelationships;

    protected $table = 'regioes';
    protected $fillable = ['nome'];

    public function estados(){
        return $this->hasMany(Estado::class);
    }

    public function fornecedores(){
        return $this->hasManyThrough(Fornecedor::class, Estado::class);
    }

    public function produtos()
    {
        return $this->hasManyDeep(
            Produto::class,
            [Estado::class,Fornecedor::class]
        );
    }
}
```

Agora podemos acessar os relacionados a uma região específica:

```
Regiao::where('nome','Sul')→with('produtos')→get();
```

Equivaleria a:

```
Produto::whereHas('fornecedor',  
    fn($q)⇒  
        $q→whereHas('estado',fn($q)⇒  
            $q→whereHas('regiao',  
                fn($q)⇒  
                    $q→where('nome','like','Sul'))))  
→get()
```

Outro pacote interessante de <https://github.com/staudenmeir> é o **belongs-to-through**, ou seja o inverso do ***hasManyThrough*** com vários níveis.

Produto→*belongsTo*→**Fornecedor**→*belongsTo*→**Estado**→*belongsTo*→**Região**

Vamos utilizar na classe Produto para relacionar os produtos com as regiões.

```
<?php
```

```
namespace App\Models;
```

```
use Illuminate\Database\Eloquent\Factories\HasFactory;  
use Illuminate\Database\Eloquent\Model;
```

```
class Produto extends Model
```

```
{
```

```
    use HasFactory, \Znck\Eloquent\Traits\BelongsToThrough;
```

```
    protected $fillable = [  
        "nome",  
        "descricao",  
        "preco",  
        "qtd_estoque",  
        "importado"
```

```

];

public function fornecedor(){
    return $this->belongsTo(Fornecedor::class);
}

public function regioao(){
    return $this->belongsToThrough(
        Regiao::class,
        [
            Estado::class,
            Fornecedor::class
        ],
        null,
        [
            Regiao::class=>'regiao_id',
            Fornecedor::class=>'fornecedor_id'
        ]
    );
}
}

```

Poderemos agora trazer a região de um Produto da seguinte forma:

```

Produto::with('regiao')->where('preco','<',500)->get()->first();

```

Neste caso, o **first()** foi utilizado para retornar o primeiro objeto da coleção apenas.

Exemplo para somar o total de produtos da região Sul e Sudeste antes do pacote **belongs-to-through**:

```

Produto::whereHas('fornecedor',fn($q)->$q->whereHas('estado',fn($q)->$q->
    whereHas('regiao',fn($q)->$q->where('nome','like','%su%'))))>count()

```

Com o pacote ***belongs-to-through***:

```
Produto::whereHas('regiao',fn($q)⇒$q→where('regioes.nome','like','%su%'))->count()
```

SQL: QueryLog

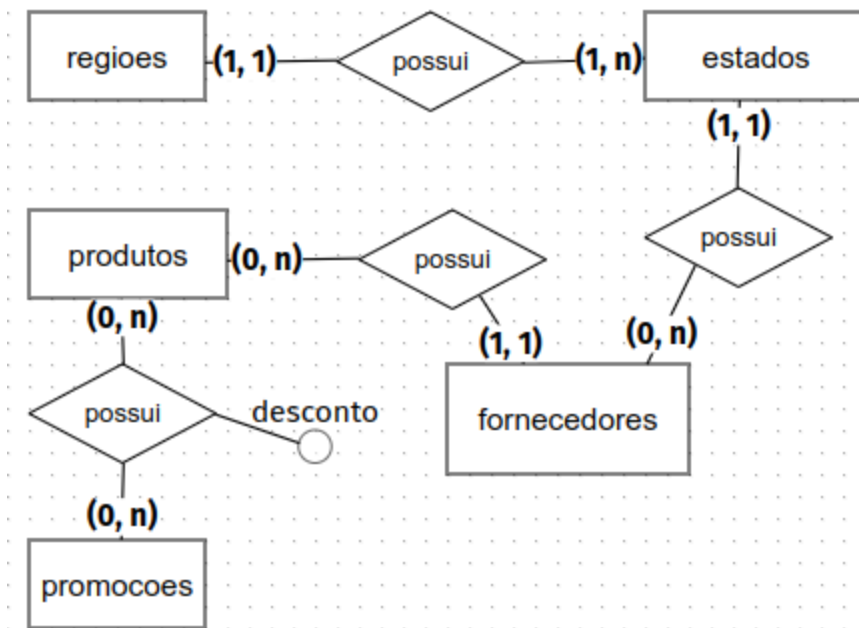
```
select count(*) as aggregate from `produtos` where exists (select * from `regioes` inner join `estados` on `estados`.`regiao_id` = `regioes`.`id` inner join `fornecedores` on `fornecedores`.`estado_id` = `estados`.`id` where `fornecedores`.`id` = `produtos`.`fornecedor_id` and `regioes`.`nome` like '%su%')
```

Apenas tivemos que adicionar o nome da tabela **"regioes"** antes da coluna de nome para que a mesma não ficasse ambígua no SQL final.

Relação Many-to-Many (N:N)

A relação N:N é representada no **Eloquent** através do método ***belongsToMany*** nas duas models que fazem parte to relacionamento. Para este exemplo, iremos criar a relação de Produto com Promoção, considerando que um Produto poderá possuir diversas promoções ao longo do ano de acordo com datas comerciais, como Dia das Mães, Black Friday, Natal e etc...

A Figura abaixo representa o banco de dados criado até o momento, adicionamos agora apenas a relação N:N de Produto com Promoção, com o atributo desconto na tabela pivô.



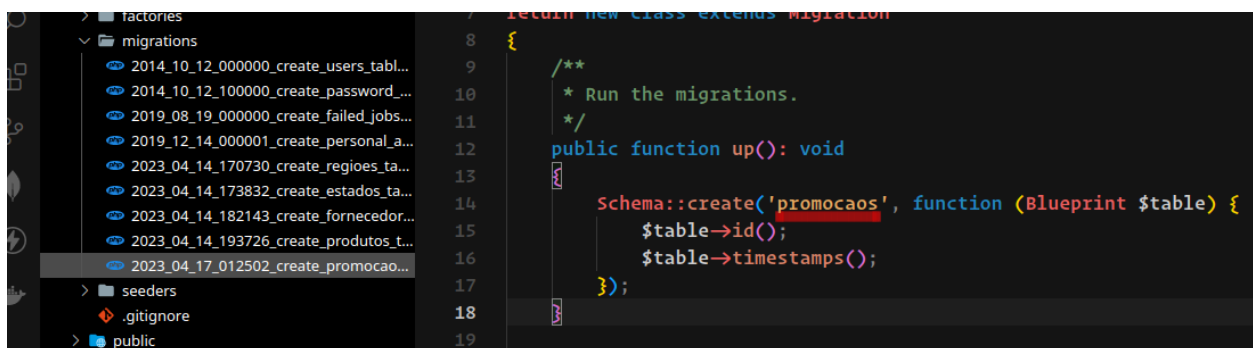
Para criar o modelo de promoção iremos usar o comando do artisan `make:model` :

```
php artisan make:model Promocao -ms
```

Criamos novamente uma **migration (-m)** e um **seeder (-s)** para o modelo Promoção.

MIGRATIONS:

Após criados os arquivos da **model**, **migration** e **seeder**, é preciso observar que a **migration create_promocaos_table** segue o padrão de nomes do **Eloquent**, o qual devemos alterar. Mais a frente, também corrigiremos a **model** através da propriedade `$table`.



Alterar o nome da **migration** é opcional, mais importante é alterar o nome da **tabela** nas funções **up()** e **down()**. No exemplo abaixo, criamos uma propriedade privada para definir o nome da tabela já na inicialização desta propriedade (**\$tableName**), adicionamos também as definições de esquema da tabela, veja o código completo abaixo:

```
<?php

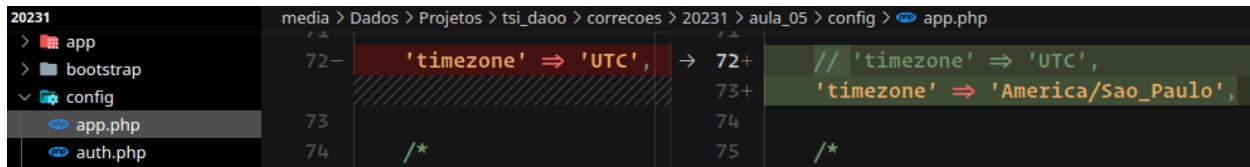
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    private $tableName = 'promocoes';
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create($this->tableName, function (Blueprint $table) {
            $table->id();
            $table->dateTime('inicio');
            $table->dateTime('fim');
            $table->string('nome');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists($this->tableName);
    }
}
```

```
}
};
```

Notem que iremos trabalhar com datas, conforme os campos início e fim, ambos do tipo **dateTime**. No entanto, antes de seguirmos com as configurações do banco, vamos aproveitar para definir a configuração do TimeZone da aplicação. Para tanto, alteraremos o arquivo **config/app.php**, conforme a imagem abaixo:



```
'timezone' => 'America/Sao_Paulo',
```

Criamos também a tabela **produto_promocao**, pivô da relação N:N, com o comando do **artisan** `make:migration` :

```
php artisan make:migration create_produto_promocao_table
```

Assim criamos a seguinte estrutura na **migration** da tabela **produto_promocao**:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
```

```

Schema::create('produto_promocao', function (Blueprint $table) {
    $table->foreignId('produto_id')
        ->constrained()
        ->onDelete('cascade');
    $table->foreignId('promocao_id')
        ->references('id')->on('promocoes')
        ->cascadeOnDelete();
    $table->primary(['produto_id','promocao_id']);
    $table->integer('desconto')->default(10);
    $table->timestamps();
});

/**
 * Reverse the migrations.
 */
public function down(): void
{
    Schema::dropIfExists('promocao_produto');
}
};

```

Observe a criação da chave composta primária com o método **primary()** recebendo como parâmetro um array com o nome das chaves estrangeiras (`['produto_id','promocao_id']`).

MODELS:

Agora que já temos as estruturas das tabelas, iremos criar as relações em cada model com o método **belongsToMany**. O método deverá ser chamado em ambos os modelos. Iniciaremos pela configuração da model **Promocao**, alterando a classe `app/Models/Promocao.php` e incluindo os campos adequados na propriedade **\$fillable**, além da propriedade padrão do **Eloquent** para definir o nome da tabela, **\$table**, e finalmente a relação de Promocao com Produtos através do método **belongsToMany**.

Classe completa da model `App/Models/Promocao.php`

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Promocao extends Model
{
    protected $table = "promocoes";
    protected $fillable = [
        'nome',
        'inicio',
        'fim',
        'desconto'
    ];

    public function produtos()
    {
        return $this->belongsToMany(Produto::class);
    }
}

```

Alteração da modelo App/Models/**Produto**.php, adicionando o método ***promocoes***.

```

public function promocoes()
{
    return $this->belongsToMany(Promocao::class);
}

```

SEEDERS:

PromocaoSeeder

Com a estrutura já definida, assim como as models, partimos para os seeders relacionados com o exemplo N:N de produto e promoção. Portanto, iniciaremos

pela classe **PromocaoSeeder** localizada em database/seeder/**PromocaoSeeder**.php para registrar algumas promoções pré-definidas no banco. Neste exemplo, definimos cinco promoções padrões direto no código (**hard coded**), sendo elas as promoções de Carnaval, Dia das Mães, Black Friday, Cyber Monday e Natal:

```
<?php

namespace Database\Seeders;

use App\Models\Promocao;
use Carbon\Carbon;
use Exception;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Log;

class PromocaoSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        try {
            $createdAt = Carbon::now()→toDateTimeString();
            $promocoos = [
                [
                    'nome'    ⇒ 'Dia das Mães',
                    'inicio'  ⇒ Carbon::create(null, 05, 4, 0, 0, 0)→toDateTimeString
(),
                    'fim'     ⇒ Carbon::create(null, 05, 15, 20, 0, 0)→toDateTimeStrin
g(),
                    'created_at'⇒ $createdAt,
                    'updated_at'⇒ $createdAt
                ],
                [
```

```

        'nome'    => 'Black Friday',
        'inicio' => Carbon::create(null, 11, 14, 8, 0, 0)→toDateTimeString
    ),
        'fim'     => Carbon::create(null, 11, 25, 0, 0, 0)→toDateTimeString
    ),
        'created_at' => $createdAt,
        'updated_at' => $createdAt,
    ],
    [
        'nome'    => 'Cyber Monday',
        'inicio' => Carbon::create(null, 11, 25, 13, 0, 0)→toDateTimeString
    g(),
        'fim'     => Carbon::create(null, 11, 30, 0, 0, 0)→toDateTimeString
    ),
        'created_at' => $createdAt,
        'updated_at' => $createdAt,
    ],
    [
        'nome'    => 'Natal',
        'inicio' => Carbon::create(null, 12, 20, 8, 0, 0)→toDateTimeString
    ),
        'fim'     => Carbon::create(null, 12, 26, 0, 0, 0)→toDateTimeString
    ),
        'created_at' => $createdAt,
        'updated_at' => $createdAt,
    ],
];

$date = Carbon::create(null, 02, 8, 12, 0, 0);
$formatDtFim = $date→toDateTimeString();
$formatDtInicio = $date→subDays(7)→toDateTimeString();
$promocoos[] = [
    'nome'    => 'Carnaval',
    'inicio' => $formatDtInicio,
    'fim'     => $formatDtFim,
    'createdAt' => $createdAt,

```

```

        'updated_at' => $createdAt,
    ];

    Promocao::insert($promocoes)
        ? Log::channel('stderr')->info("Promocoes criadas!")
        : throw new Exception("Erro ao criar promoções!");

    } catch(Exception $error) {
        Log::debug($error->getMessage());
        Log::channel('stderr')->error($error->getMessage());
    }
}
}

```

A classe **Carbon**, do pacote `nesbot/carbon`, facilita o trabalho com datas no PHP, seria como a antiga **Moment** ou seu fork, **Luxon**, no Javascript, nelas são abstraídas várias operações temporais das implementações nativas da linguagem. No PHP, a Carbon estende diretamente a classe padrão **DateTime**. No exemplo de código acima usamos os métodos ***create()***, ***now()***, ***subDays()*** e ***toDateTimeString()***.

- ***now()***: cria um novo objeto de data baseado na data e no tempo atual.
- ***create()***: cria um nova data, podendo passar os valores de ano, mês, dia e hora como parâmetros, sendo que quando omitimos um deles, ou passamos ***null***, o objeto é criado com a a data ou tempo atual.
- ***toDateTimeString()***: retorna a data como string e formatado para o padrão de acordo com o TimeZone configurado.
- ***subDays()***: altera o objeto de data decrementado em dias de acordo com o parâmetro passado.

O Carbon ainda possui a operação de adição com o método ***addDays()***, assim como operações com anos, meses e etc...; Descubra mais facilidades do Carbon na sua **documentação** própria.

Exemplos para rodar no tinker (`php artisan tinker`):

```

> use Carbon\Carbon;
> Carbon::now()
= Carbon\Carbon @1682184657 {#4726
  date: 2023-04-22 14:30:57.555332 America/Sao_Paulo (-03:00),
}
> use Carbon\Carbon;
> $now = Carbon::now()
= Carbon\Carbon @1682184987 {#3753
  date: 2023-04-22 14:36:27.025091 America/Sao_Paulo (-03:00),
}

> echo $now;
2023-04-22 14:36:27↵
> Carbon::setToStringFormat('d-m-Y H:i:s')
= null

> echo $now;
22-04-2023 14:36:27↵
> Carbon::setToStringFormat('d/m/Y')
= null

> echo $now;
22/04/2023↵
> $now->addMonth(3)
= Carbon\Carbon @1690047387 {#3753
  date: 2023-07-22 14:36:27.025091 America/Sao_Paulo (-03:00),
}

> echo $now;
22/07/2023↵

```

Outro item novo no código da **PromocaoSeeder** é o uso do método estático **debug** da classe facade Log, no entanto, este método gera uma saída no arquivo **storage/logs/laravel.log**, padrão para logar mensagens de erro. Por outro lado, o

método **`channel('stderr')`** envia para a saída padrão de erros do php (`php://stderr`), ou seja, aparece na "tela" do terminal.

ProdutoPromocaoSeeder

Para criar a classe seed da relação Produto e Promocao usaremos o comando do artisan `make:seed` :

```
php artisan make:seeder ProdutoPromocaoSeeder
```

Este comando criará o arquivo `database/seeder/ProdutoPromocaoSeeder.php` o qual iremos editar para incluir um algoritmo de criação de relações entre Produto e Promocao, populando a tabela pivo **`produto_promocao`**. Com o método **`attach`** do Eloquent, e com as relações devidamente configuradas, podemos facilmente "anexar" (**`attach`**) uma nova Promocao a uma model Produto:

```
$produto = new Produto::find(1);  
$produto->promocoas()->attach(3)
```

No exemplo acima, estamos cadastrando na tabela pivo, a promoção de ID = 3 como nova promoção relacionada ao Produto de ID=1. Também é possível repassar um array de IDs para o método **`attach`**, como fazemos no exemplo da classe ***ProdutoPromocaoSeeder*** abaixo:

```
<?php  
  
namespace Database\Seeders;  
  
use App\Models\Produto;  
use App\Models\Promocao;  
use Carbon\Carbon;  
use Exception;  
use Illuminate\Support\Facades\Log;  
use Illuminate\Database\Seeder;  
  
class PromocaoProdutoSeeder extends Seeder  
{
```

```

/**
 * Run the database seeds.
 */
public function run(): void
{
    $listProdutos = Produto::all();
    $totalPromocao = Promocao::count();

    if(!$totalPromocao || !$listProdutos->count()) {
        throw new Exception('Error: registrar produtos e/ou promocoos!');
    }

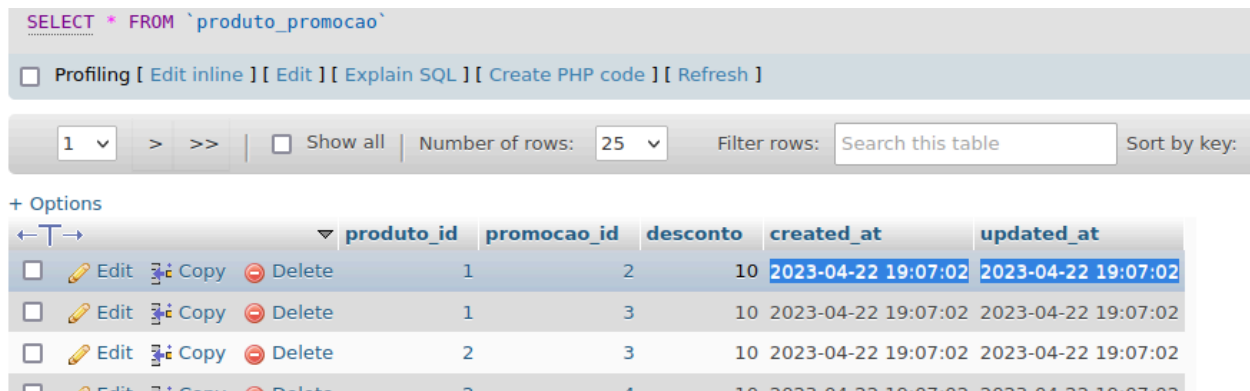
    $now = Carbon::now()->toDateTimeString();
    $createUpdateAt = ['created_at'=>$now,'updated_at'=>$now];

    Log::channel('stderr')->info('Relacionando promocoos e produtos...');
    $listPromocoosIDs = Promocao::all()->pluck('id');
    $listProdutos->each(function ($produto) use ($listPromocoosIDs, $createUpdateAt) {
        [$promold,$promold2] = $listPromocoosIDs->random(2);
        $produto->promocoos()->attach([
            $promold => $createUpdateAt,
            $promold2 => $createUpdateAt,
        ]);
        Log::channel('stderr')->info("Produto: $produto->id | Promocoos: ( $promold , $promold2 )");
    });
}

```

Além de repassar ao método **attach** um array de IDs a serem “anexados” ao model, podemos repassar um array associativo (chave/valor) com os campos da tabela pivô e o valor que queremos passar para estes campos. Neste exemplo, criamos também os valores dos campos ***created_at*** e ***updated_at*** na tabela ***produto_promocao***, poderíamos repassar também um valor aleatório para o

campo desconto. Ao executar este **seeder** teríamos algo parecido com a imagem abaixo no banco de dados:



	produto_id	promocao_id	desconto	created_at	updated_at
☐ Edit ☐ Copy ☐ Delete	1	2	10	2023-04-22 19:07:02	2023-04-22 19:07:02
☐ Edit ☐ Copy ☐ Delete	1	3	10	2023-04-22 19:07:02	2023-04-22 19:07:02
☐ Edit ☐ Copy ☐ Delete	2	3	10	2023-04-22 19:07:02	2023-04-22 19:07:02
☐ Edit ☐ Copy ☐ Delete	2	4	10	2023-04-22 19:07:02	2023-04-22 19:07:02

Na documentação encontramos outros métodos para manipular registros entre relações **ManyToMany**, como **detach**, **sync** e **toggle** :

1. **detach**: desativa a relação com os IDs passados como parâmetros.
2. **sync**: ativa todos os IDs passados como parâmetros e desativa aqueles que não existem no array.
3. **toggle**: recebe um array com os IDs e ativa aqueles que não existem e desativa os já existentes.

Veja os exemplos [neste link](#).

Para que os dois **seeders** criados sejam efetivamente executados, precisamos atualizar a classe **DatabasesSeeder**:

```
<?php

namespace Database\Seeders;

// use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use App\Models\Fornecedor;
use Illuminate\Support\Facades\Log;
```

```

class DatabaseSeeder extends Seeder
{
    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        // \App\Models\User::factory(10)→create();

        // \App\Models\User::factory()→create([
        //     'name' ⇒ 'Test User',
        //     'email' ⇒ 'test@example.com',
        // ]);

        $this→call([
            RegiaoSeeder::class,
            Estado::class
        ]);

        Log::channel('stderr')→info('Criando Fornecedores com produtos...');
        Fornecedor::factory(10)
            →hasProdutos(25)
            →create();

        $this→call([
            PromocaoSeeder::class
            PromocaoProdutoSeeder::class
        ]);
    }
}

```

Para gerar os resultados no banco, executamos o seguinte comando do artisan , que irá refazer as tabelas (**fresh**) e rodar o seeder (**--seed**):

```
php artisan migrate:fresh --seed
```

O log deverá ser semelhante as imagens abaixo e o banco de dados deverá ser populado com todas as informações incluindo as relações entre Produtos e Promoções:

```
[2023-04-22 19:48:02] local.INFO: Produto: 226 | Promocoes: ( 5 , 4 )
[2023-04-22 19:48:02] local.INFO: Produto: 227 | Promocoes: ( 2 , 3 )
[2023-04-22 19:48:02] local.INFO: Produto: 228 | Promocoes: ( 5 , 4 )
[2023-04-22 19:48:02] local.INFO: Produto: 229 | Promocoes: ( 4 , 5 )
[2023-04-22 19:48:02] local.INFO: Produto: 230 | Promocoes: ( 2 , 3 )
[2023-04-22 19:48:02] local.INFO: Produto: 231 | Promocoes: ( 3 , 4 )
[2023-04-22 19:48:02] local.INFO: Produto: 232 | Promocoes: ( 1 , 2 )
[2023-04-22 19:48:02] local.INFO: Produto: 233 | Promocoes: ( 3 , 4 )
[2023-04-22 19:48:02] local.INFO: Produto: 234 | Promocoes: ( 4 , 5 )
[2023-04-22 19:48:02] local.INFO: Produto: 235 | Promocoes: ( 1 , 2 )
[2023-04-22 19:48:02] local.INFO: Produto: 236 | Promocoes: ( 5 , 4 )
[2023-04-22 19:48:02] local.INFO: Produto: 237 | Promocoes: ( 2 , 3 )
[2023-04-22 19:48:02] local.INFO: Produto: 238 | Promocoes: ( 1 , 2 )
[2023-04-22 19:48:02] local.INFO: Produto: 239 | Promocoes: ( 3 , 4 )
[2023-04-22 19:48:02] local.INFO: Produto: 240 | Promocoes: ( 5 , 4 )
[2023-04-22 19:48:02] local.INFO: Produto: 241 | Promocoes: ( 4 , 5 )
[2023-04-22 19:48:02] local.INFO: Produto: 242 | Promocoes: ( 2 , 3 )
[2023-04-22 19:48:02] local.INFO: Produto: 243 | Promocoes: ( 4 , 5 )
[2023-04-22 19:48:02] local.INFO: Produto: 244 | Promocoes: ( 4 , 5 )
[2023-04-22 19:48:02] local.INFO: Produto: 245 | Promocoes: ( 4 , 5 )
[2023-04-22 19:48:02] local.INFO: Produto: 246 | Promocoes: ( 1 , 2 )
[2023-04-22 19:48:02] local.INFO: Produto: 247 | Promocoes: ( 4 , 5 )
[2023-04-22 19:48:02] local.INFO: Produto: 248 | Promocoes: ( 5 , 4 )
[2023-04-22 19:48:02] local.INFO: Produto: 249 | Promocoes: ( 3 , 4 )
[2023-04-22 19:48:02] local.INFO: Produto: 250 | Promocoes: ( 1 , 2 )
```

CONSULTAS N:N

Iremos utilizar o artisan tinker para realizar algumas consultas através do Eloquent e do Query/Builder para os models Produto e Promocao, portanto executamos o seguinte comando para abrir o **tinker**:

```
php artisan tinker
```

```
...correcoes/20231/aula_05 <?php 8.2.5 > correcao-aula-08-c
$ php artisan tinker
Psy Shell v0.11.14 (PHP 8.2.5 - cli) by Justin Hileman
>
```

No **tinker** começamos importando as classes necessárias:

```
$ php artisan tinker
Psy Shell v0.11.14 (PHP 8.2.5 — cli) by Justin Hileman
```

```
> use App\Models\Produto
> use App\Models\Promocao
>
```

Vamos listar promoções do Produto de ID=1, apenas acessando a propriedade ***promocoos***:

```
> Produto::find(1)→promocoos
= Illuminate\Database\Eloquent\Collection {#4704
  all: [
    App\Models\Promocao {#4706
      id: 4,
      inicio: "2023-12-20 08:00:00",
      fim: "2023-12-26 00:00:00",
      nome: "Natal",
      created_at: "2023-04-22 20:33:45",
      updated_at: "2023-04-22 20:33:45",
      pivot: Illuminate\Database\Eloquent\Relations\Pivot {#4703
        produto_id: 1,
        promocao_id: 4,
      },
    },
    App\Models\Promocao {#4707
      id: 5,
      inicio: "2023-02-01 12:00:00",
      fim: "2023-02-08 12:00:00",
      nome: "Carnaval",
      created_at: "2023-04-22 20:33:45",
      updated_at: "2023-04-22 20:33:45",
      pivot: Illuminate\Database\Eloquent\Relations\Pivot {#4702
        produto_id: 1,
        promocao_id: 5,
      },
    },
  ],
}
```

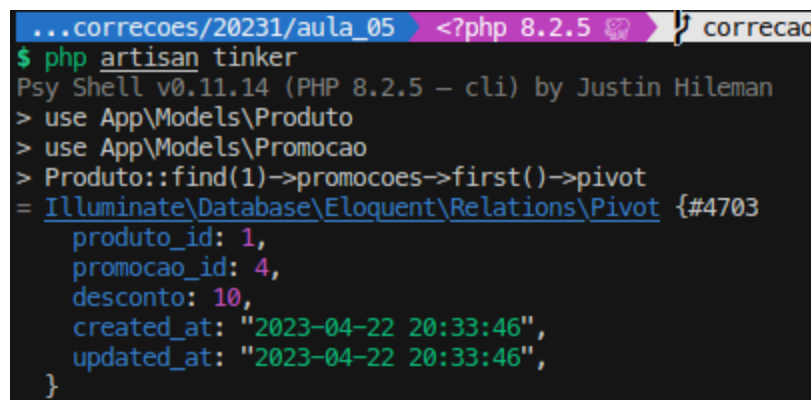
```
],  
}
```

Veja que podemos acessar diretamente a **Eloquent/Collection** através do nome da relação apenas, que deverá possuir o mesmo nome do método criado na model de Produto.

Observe nos resultados do exemplo anterior a propriedade **pivot**, esta propriedade, relacionada ao modelo **Promocao**, representa a tabela pivô da relação (**produto_promocao**). No entanto não estamos acessando os dados das colunas de **timestamps** e **desconto**. Para corrigir, adicionamos a seguinte alteração na classe model de **Produto**:

```
public function promocoos()  
{  
    return $this->belongsToMany(Promocao::class)  
        ->withPivot('desconto')  
        ->withTimestamps();  
}
```

Desta forma é possível acessar os atributos da tabela intermediária, é necessário **reiniciar o tinker** para atualizar os **modelos em memória**.



```
...correcoes/20231/aula_05 <?php 8.2.5 > correcao-  
$ php artisan tinker  
Psy Shell v0.11.14 (PHP 8.2.5 - cli) by Justin Hileman  
> use App\Models\Produto  
> use App\Models\Promocao  
> Produto::find(1)->promocoos->first()->pivot  
= Illuminate\Database\Eloquent\Relations\Pivot {#4703  
    produto_id: 1,  
    promocao_id: 4,  
    desconto: 10,  
    created_at: "2023-04-22 20:33:46",  
    updated_at: "2023-04-22 20:33:46",  
}
```

É necessário atualizar o modelo **Promocao** com os campos **desconto** e **timestamp**, caso contrário não conseguiremos acessar também pela consulta através do model **Promocao**, como mostra a imagem abaixo:

```

...correcoes/20231/aula_05 <?php 8.2.5 } correcao-aula-0
$ php artisan tinker
Psy Shell v0.11.14 (PHP 8.2.5 - cli) by Justin Hileman
> use App\Models\Promocao
> Promocao::find(4)->produtos->first()->pivot
= Illuminate\Database\Eloquent\Relations\Pivot {#4843
    promocao_id: 4,
    produto_id: 1,
}
>

```

Correção:

```

public function produtos()
{
    return $this->belongsToMany(Produto::class)
        ->withPivot('desconto')
        ->withTimestamps();
}

```

Resultado:

```

...correcoes/20231/aula_05 <?php 8.2.5 } correcao-
$ php artisan tinker
Psy Shell v0.11.14 (PHP 8.2.5 - cli) by Justin Hileman
> use App\Models\Produto
> use App\Models\Promocao
> Promocao::find(4)->produtos->first()->pivot
= Illuminate\Database\Eloquent\Relations\Pivot {#4856
    promocao_id: 4,
    produto_id: 1,
    desconto: 10,
    created_at: "2023-04-22 20:33:46",
    updated_at: "2023-04-22 20:33:46",
}

```

É possível acessar diretamente a relação chamando o método **produtos()**, por exemplo, no lugar da propriedade **produtos**. A propriedade com o nome da relação traz a coleção ou o modelo relacionado, já a relação é um objeto que pode ser usado para construir uma consulta.

```
> Promocao::find(4)->produtos()
= Illuminate\Database\Eloquent\Relations\BelongsToMany {#5617
  +withTimestamps: true,
}
```

A partir do objeto do tipo **BelongsToMany** pode-se usar métodos do **Query/Builder**, como por exemplo o **find()**. No exemplo a seguir procuramos o produto de ID=1 mas que possui relação com promoção:

```
> \DB::enableQueryLog()
= null

> \DB::getQueryLog();
= []

> Promocao::find(4)->produtos()->find(1)
= App\Models\Produto {#5971
  id: 1,
  fornecedor_id: 1,
  nome: "Et veniam voluptatem iure.",
  descricao: "Ducimus aut facilis deserunt et sint pariatur qui itaque molestia
e consequuntur nisi.",
  preco: 9756.59,
  qtd_estoque: 103,
  importado: 1,
  created_at: "2023-04-22 20:33:45",
  updated_at: "2023-04-22 20:33:45",
  pivot: Illuminate\Database\Eloquent\Relations\Pivot {#5975
    promocao_id: 4,
    produto_id: 1,
    desconto: 10,
    created_at: "2023-04-22 20:33:46",
    updated_at: "2023-04-22 20:33:46",
  },
}
```

```

> \DB::getQueryLog();
= [
  [
    "query" ⇒ "select * from `promocoas` where `promocoas`.`id` = ? limit 1",
    "bindings" ⇒ [
      4,
    ],
    "time" ⇒ 1.21,
  ],
  [
    "query" ⇒ "select `produtos`.*, `produto_promocao`.`promocao_id` as `pivot_promocao_id`, `produto_promocao`.`produto_id` as `pivot_produto_id`, `produto_promocao`.`desconto` as `pivot_desconto`, `produto_promocao`.`created_at` as `pivot_created_at`, `produto_promocao`.`updated_at` as `pivot_updated_at` from `produtos` inner join `produto_promocao` on `produtos`.`id` = `produto_promocao`.`produto_id` where `produto_promocao`.`promocao_id` = ? and `produtos`.`id` = ? limit 1",
    "bindings" ⇒ [
      4,
      1,
    ],
    "time" ⇒ 1.18,
  ],
]

```

Observe as duas consultas geradas, a primeira para buscar a Promocao de ID=1, já a segunda busca o Produto de ID=1 relacionado com a Promocao ID=4.

Qual a diferença entre estas chamadas?

- 1) Promocao::find(4)→produtos()→find(1)
- 2) Promocao::find(4)→produtos→find(1)

O primeiro acessa a relação, ou seja, o retorno do método produtos() na classe Promocao, belongsToMany. Já o segundo, retorna a coleção de produtos relacionados a class Promocao.

Referências:

- Relações Muito-para-muitos
 - <https://laravel.com/docs/10.x/eloquent-relationships#many-to-many>
 - <https://laravel-news.com/effective-eloquent>
- Carbon
 - <https://carbon.nesbot.com/docs/>

LICENÇA:

Creative Commons — Attribution-ShareAlike 4.0 International
— CC BY-SA 4.0